Relatório Técnico: Aplicação do Teorema Mestre na Análise de Algoritmos Recursivos

1. Introdução

O objetivo deste trabalho é aplicar o **Teorema Mestre** na determinação da complexidade assintótica de três algoritmos recursivos distintos de Divisão e Conquista, validar matematicamente os resultados por meio de **indução matemática** (método da substituição) e confrontar as previsões teóricas com dados experimentais extraídos da execução dos algoritmos na linguagem C.

2. Revisão Teórica

O Teorema Mestre resolve relações de recorrência da forma $T(n) = aT(n/b) + f(n)$, onde $a \geq 1$ é o número de subproblemas, $b > 1$ é o fator de redução da entrada e $f(n)$ é o custo do trabalho externo às chamadas recursivas. Para compreender o teorema, imagine a execução do algoritmo como uma árvore invertida: a 'raiz' é a primeira chamada da função com todo o vetor $f(n)$, os galhos são as divisões da recursão, e as 'folhas' representam o caso base $n^{\log_b a}$, onde o vetor não pode mais ser dividido. Sendo assim, a solução assintótica baseia-se na comparação entre $f(n)$ e o número de folhas da árvore de recursão, dado por $n^{\log_b a}$:

1. **Caso 1 (Recursividade Dominante):** Se $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ para alguma constante $\epsilon > 0$, o custo concentra-se nas folhas e $T(n) = \Theta\left(n^{\log_b a}\right)$.

2. **Caso 2 (Custo Equilibrado):** Se $f(n) = \Theta\left(n^{\log_b a}\right)$, o trabalho é distribuído uniformemente entre os níveis da árvore e $T(n) = \Theta\left(n^{\log_b a} \log n\right)$.

3. **Caso 3 (Custo Externo Dominante):** Se $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ para $\epsilon > 0$ e a condição de regularidade $af(n/b) \leq kf(n)$ for satisfeita para algum $k < 1$, o custo da raiz domina e $T(n) = \Theta\left(f(n)\right)$.

3. Metodologia

- **Objetivo Geral e Específicos:** Validar experimentalmente os três casos do Teorema Mestre implementando três programas independentes em C que atendam às restrições do enunciado ($a \neq 2$ nos Casos 1 e 2, e $a \neq b$ em pelo menos um deles):

    - **Problema 1 (problema1_caso1.c):** $T(n) = 4T(n/2) + n \Longrightarrow$ Caso 1 ($a = 4 \neq 2$ e $a \neq b$).

    - **Problema 2 (problema2_caso2.c):** $T(n) = 3T(n/3) + n \Longrightarrow$ Caso 2 ($a = 3 \neq 2$).

    - **Problema 3 (problema3_caso3.c):** $T(n) = 2T(n/2) + n^2 \Longrightarrow$ Caso 3 ($a = 2, b = 2$).

- **Ferramentas Utilizadas:** Compilador **GCC (Linguagem C)** com a biblioteca <time.h> (clock() e CLOCKS_PER_SEC) para medição de tempo e ponteiros acumuladores (long long *operacoes) para contagem exata de operações. Para análise e geração dos gráficos, utilizou-se **Python 3** (pandas, numpy e matplotlib).

- **Métodos de Coleta e Análise:** Cada tamanho de entrada N foi executado por REPETICOES = 20 iterações internas utilizando uma variável global volatile (sink) para impedir otimizações de código morto pelo compilador. Cada executável foi rodado 3 vezes completas, registrando os resultados de forma unificada no arquivo resultados_teorema_mestre.csv em modo *append* ("a+"). Nos Problemas 1 e 3, testaram-se potências de 2 ($N \in [16,16384]$); no Problema 2, potências de 3 ($N \in [27,1594323]$). As curvas teóricas foram ajustadas aos tempos em milissegundos por mínimos quadrados ($c \cdot g(n)$).

- **Justificativa das Escolhas:** O uso de potências exatas de b (2^k e 3^k) evita truncamentos de divisão inteira em C, permitindo comprovar a igualdade exata entre o contador empírico de operações e a fórmula provada por

indução. A escolha de $a = 4, b = 2$ no Problema 1 garante um expoente inteiro ($\log_2 4 = 2$), tornando a indução algébrica exata.

- **Limitações da Metodologia:** A função clock() possui resolução limitada em entradas muito pequenas ($N \leq 64$ nos Problemas 1 e 3; $N \leq 2187$ no Problema 2), registrando $0{,}000$ ms e omitindo esses pontos iniciais na escala logarítmica. Além disso, os tempos reais sofrem influência de cache L1/L2 e sobrecarga de pilha (*call stack*) do sistema operacional.

4. Análise Teórica

Nesta seção, apresenta-se a análise teórica individual dos três algoritmos implementados. Para cada problema, exibe-se o trecho essencial do código-fonte em C que origina a relação de recorrência, seguido pela aplicação explícita do **Teorema Mestre** e pela confirmação formal via **Indução Matemática**.

4.1 Problema 1 — Caso 1: Recursividade Dominante (problema1_caso1.c)

- **Descrição do Problema e Código-Fonte Essencial:** O algoritmo realiza uma varredura linear sobre um segmento de tamanho n e, em seguida, subdivide o processamento em 4 chamadas recursivas sobre metades do vetor ($n/2$). Abaixo, destaca-se o núcleo da função resolver_problema1_caso1 responsável por gerar a recorrência:

Código:

C
```
long long resolver_problema1_caso1(const int *vetor, int n, long long *operacoes) {
    if (n <= 1) {              // Caso base: T(1) = 1
        (*operacoes)++;
        return vetor[0];
    }

    long long soma_local = 0;
    for (int i = 0; i < n; i++) { // Trabalho fora da recursão: f(n) = n
        (*operacoes)++;
        soma_local += vetor[i];
    }

    int metade = n / 2;         // Divisão da entrada: b = 2
    // Passo recursivo: a = 4 chamadas de tamanho n/2 -> 4T(n/2)
    long long r1 = resolver_problema1_caso1(vetor, metade, operacoes);
    long long r2 = resolver_problema1_caso1(vetor + metade, metade, operacoes);
```

```
    long long r3 = resolver_problema1_caso1(vetor, metade, operacoes);
    long long r4 = resolver_problema1_caso1(vetor + metade, metade, operacoes);

    return soma_local + r1 + r2 + r3 + r4;
}
```

- **Recorrência Analisada:** A partir da inspeção direta do código acima, o laço for executa n operações e a função realiza 4 chamadas recursivas para entradas de tamanho $n/2$, resultando na recorrência:

$$T(n) = 4T\left(\frac{n}{2}\right) + n, \quad \text{com } T(1) = 1$$

- **Identificação dos Parâmetros ($a, b, f(n)$) e Cálculo de $n^{\log_b a}$: Usando o Teorema Mestre**, comparamos a recorrência com o formato padrão $T(n) = aT(n/b) + f(n)$ e extraímos os seguintes parâmetros:

  - $a = 4$ (quatro subproblemas recursivos por chamada, atendendo às restrições $a \neq 2$ e $a \neq b$);

  - $b = 2$ (fator de redução da entrada ao meio a cada nível);

  - $f(n) = n = \Theta(n^1)$ (custo linear do laço for executado fora da recursão).

  Aplicando a fórmula da função polinomial das folhas do **Teorema Mestre**, temos:

$$n^{\log_b a} = n^{\log_2 4} = n^2$$

- **Enquadramento e Justificativa pelo Teorema Mestre: Aplicando o Teorema Mestre**, confrontamos o custo externo $f(n) = n^1$ com o peso das folhas $n^{\log_2 4} = n^2$. Verifica-se que $f(n)$ é polinomialmente menor que $n^{\log_b a}$, pois para a constante $\epsilon = 1 > 0$ cumpre-se a condição:

$$f(n) = n^1 = O\left(n^{\log_2 4 - \epsilon}\right) = O(n^{2-1}) = O(n^1)$$

Portanto, pelo **Teorema Mestre**, o problema enquadra-se no **Caso 1 (Recursividade Dominante: o custo concentrado nas folhas da árvore de recursão domina assintoticamente o trabalho realizado nos níveis internos)**.

- **Complexidade Assintótica Obtida:** Pelo **Caso 1 do Teorema Mestre (Recursividade Dominante)**, conclui-se que a complexidade assintótica do algoritmo é:

$$T(n) = \Theta\left(n^{\log_b a}\right) = \Theta(n^2)$$

- **Demonstração Formal por Indução Matemática:** Para confirmar o resultado obtido pelo Teorema Mestre, demonstramos por indução que a fórmula fechada exata para potências de 2 ($n = 2^k$) é $T(n) = 2n^2 - n$:

1. **Caso Base ($n = 1$):** No código-fonte, para n <= 1, executa-se 1 operação ($T(1) = 1$). Pela fórmula, $T(1) = 2(1)^2 - 1 = 1$ (Válido).

2. **Hipótese de Indução (H.I.):** Assume-se que a fórmula é válida para $n/2$, isto é: $T(n/2) = 2(n/2)^2 - (n/2)$.

3. **Passo Indutivo:** Substituindo a H.I. na recorrência original $T(n) = 4T(n/2) + n$:

$$T(n) = 4\left[2\left(\frac{n}{2}\right)^2 - \frac{n}{2}\right] + n = 4\left[2\left(\frac{n^2}{4}\right) - \frac{n}{2}\right] + n = 2n^2 - 2n + n$$
$$= 2n^2 - n$$

Como o termo de maior grau de $2n^2 - n$ é quadrático ($n^2 \leq 2n^2 - n \leq 2n^2$ para $n \geq 1$), confirma-se que $T(n) = \Theta(n^2)$, coincidindo exatamente com o resultado do Teorema Mestre.

4.2 Problema 2 — Caso 2: Custo Equilibrado (problema2_caso2.c)

- **Descrição do Problema e Código-Fonte Essencial:** O algoritmo implementa uma estratégia de divisão e conquista ternária: divide o vetor de tamanho n em três terços iguais ($n/3$), resolve recursivamente as 3 partes e realiza uma varredura linear sobre os n elementos para combinar os resultados. Abaixo está o trecho central da função resolver_problema2_caso2:

Código:

C

```
long long resolver_problema2_caso2(const int *vetor, int n, long long *operacoes) {
    if (n <= 1) {              // Caso base: T(1) = 1
        (*operacoes)++;
        return (n == 1) ? vetor[0] : 0;
    }

    int terco = n / 3;          // Divisão da entrada: b = 3
    // Passo recursivo: a = 3 chamadas de tamanho n/3 -> 3T(n/3)
    long long p1 = resolver_problema2_caso2(vetor, terco, operacoes);
    long long p2 = resolver_problema2_caso2(vetor + terco, terco, operacoes);
    long long p3 = resolver_problema2_caso2(vetor + 2 * terco, n - 2 * terco, operacoes);

    long long combinacao = 0;
    for (int i = 0; i < n; i++) { // Trabalho fora da recursão: f(n) = n
        (*operacoes)++;
        combinacao += vetor[i];
    }

    return p1 + p2 + p3 + combinacao;
}
```

- **Recorrência Analisada:** Analisando a estrutura do código acima, as 3 chamadas recursivas de tamanho $n/3$ somadas ao laço for de combinação de tamanho n determinam a recorrência:

$$T(n) = 3T\left(\frac{n}{3}\right) + n, \quad \text{com } T(1) = 1$$

- **Identificação dos Parâmetros ($a, b, f(n)$) e Cálculo de $n^{\log_b a}$: Usando o Teorema Mestre**, identificamos os parâmetros da recorrência:

  - $a = 3$ (três chamadas recursivas por nível, atendendo à restrição $a \neq 2$);

  - $b = 3$ (tamanho do subproblema dividido por 3 a cada nível);

- $f(n) = n = \Theta(n^1)$ (trabalho linear de combinação no laço for fora da recursão).

Calculando o termo polinomial associado às folhas pelo **Teorema Mestre**, obtemos:

$$n^{\log_b a} = n^{\log_3 3} = n^1 = n$$

- **Enquadramento e Justificativa pelo Teorema Mestre: Aplicando o Teorema Mestre**, comparamos $f(n) = n$ com $n^{\log_3 3} = n$. Constata-se que ambas as funções crescem exatamente na mesma ordem assintótica linear:

$$f(n) = n = \Theta\left(n^{\log_3 3}\right) = \Theta(n)$$

Portanto, pelo **Teorema Mestre**, a recorrência enquadra-se no **Caso 2 (Custo Equilibrado: o trabalho de dividir o problema em subproblemas e o trabalho de combiná-los possuem o mesmo peso assintótico, distribuindo um custo $\Theta(n)$ de maneira uniforme em cada um dos $\log_3 n$ níveis da árvore)**.

- **Complexidade Assintótica Obtida:** Pelo **Caso 2 do Teorema Mestre (Custo Equilibrado)**, multiplica-se o termo comum por $\log n$, obtendo-se a complexidade assintótica:

$$T(n) = \Theta\left(n^{\log_b a} \log n\right) = \Theta(n \log_3 n) = \Theta(n \log n)$$

- **Demonstração Formal por Indução Matemática:** Confirmamos o resultado do Teorema Mestre provando por indução que a fórmula fechada exata para potências de 3 ($n = 3^k$) é $T(n) = n \log_3 n + n$:

1. **Caso Base ($n = 1$):** No código, $T(1) = 1$. Pela fórmula, $T(1) = 1 \cdot \log_3(1) + 1 = 0 + 1 = 1$ (Válido).

2. **Hipótese de Indução (H.I.):** Assume-se que para $n/3$ vale $T(n/3) = (n/3)\log_3(n/3) + (n/3)$.

3. **Passo Indutivo:** Substituindo a H.I. na recorrência original $T(n) = 3T(n/3) + n$:

$$T(n) = 3\left[\left(\frac{n}{3}\right)\log_3\left(\frac{n}{3}\right) + \frac{n}{3}\right] + n = n\log_3\left(\frac{n}{3}\right) + n + n$$

Pela propriedade logarítmica $\log_3(n/3) = \log_3 n - \log_3 3 = \log_3 n - 1$:

$$T(n) = n(\log_3 n - 1) + 2n = n\log_3 n - n + 2n = n\log_3 n + n$$

Como o termo dominante é $n\log_3 n$ ($n\log_3 n \leq n\log_3 n + n \leq 2n\log_3 n$ para $n \geq 3$), fica demonstrado que $T(n) = \Theta(n\log n)$, em total acordo com o Teorema Mestre.

4.3 Problema 3 — Caso 3: Custo Fora da Recursão Dominante (problema3_caso3.c)

- **Descrição do Problema e Código-Fonte Essencial:** O algoritmo processa inicialmente todos os pares de elementos do segmento atual por meio de dois laços for aninhados (custo quadrático n^2) e, em seguida, realiza 2 chamadas recursivas sobre metades do vetor ($n/2$). Abaixo, apresenta-se o trecho essencial da função resolver_problema3_caso3:

Código:

C
```
long long resolver_problema3_caso3(const int *vetor, int n, long long *operacoes) {
    if (n <= 1) {              // Caso base: T(1) = 1
        (*operacoes)++;
        return vetor[0];
    }

    long long custo_pares = 0;
    // Trabalho fora da recursão: dois laços aninhados -> f(n) = n * n = n^2
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            (*operacoes)++;
```

```
            custo_pares += (vetor[i] ^ vetor[j]);
        }
    }

    int metade = n / 2;         // Divisão da entrada: b = 2
    // Passo recursivo: a = 2 chamadas de tamanho n/2 -> 2T(n/2)
    long long esq = resolver_problema3_caso3(vetor, metade, operacoes);
    long long dir = resolver_problema3_caso3(vetor + metade, metade, operacoes);

    return custo_pares + esq + dir;
}
```

- **Recorrência Analisada:** Pela inspeção do código acima, o duplo laço for realiza $n \times n = n^2$ operações e o passo recursivo efetua 2 chamadas de tamanho $n/2$, estabelecendo a recorrência:

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2, \quad \text{com } T(1) = 1$$

- **Identificação dos Parâmetros ($a, b, f(n)$) e Cálculo de $n^{\log_b a}$: Usando o Teorema Mestre**, extraímos os parâmetros da recorrência:

  - $a = 2$ (duas chamadas recursivas por ativação);
  - $b = 2$ (divisão do tamanho da entrada por 2);
  - $f(n) = n^2 = \Theta(n^2)$ (custo quadrático dos laços aninhados executados fora da recursão).

  Calculando o custo polinomial das folhas pelo **Teorema Mestre**, temos:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

- **Enquadramento e Justificativa pelo Teorema Mestre: Aplicando o Teorema Mestre**, comparamos o custo fora da recursão $f(n) = n^2$ com o custo das folhas $n^{\log_2 2} = n^1$:

1. **Dominância Polinomial:** Verifica-se que $f(n) = n^2$ cresce polinomialmente mais rápido que n^1, satisfazendo a condição para $\epsilon = 1 > 0$:

$$f(n) = n^2 = \Omega\left(n^{\log_2 2+\epsilon}\right) = \Omega(n^{1+1}) = \Omega(n^2)$$

2. **Condição de Regularidade:** Exige-se que $a \cdot f(n/b) \leq k \cdot f(n)$ para alguma constante $k < 1$. Substituindo os valores:

$$2 \cdot f\left(\frac{n}{2}\right) = 2 \cdot \left(\frac{n}{2}\right)^2 = 2 \cdot \left(\frac{n^2}{4}\right) = \frac{1}{2}n^2 \leq k \cdot n^2$$

   A desigualdade é satisfeita com igualdade exata para $k = 0{,}5 < 1$.

Portanto, pelo **Teorema Mestre**, a recorrência enquadra-se no **Caso 3 (Custo Fora da Recursão Dominante: o trabalho pesado $f(n) = n^2$ executado fora das chamadas recursivas na raiz supera assintoticamente o custo da ramificação recursiva)**.

- **Complexidade Assintótica Obtida:** Pelo **Caso 3 do Teorema Mestre (Custo Fora da Recursão Dominante)**, a complexidade assintótica é determinada diretamente por $f(n)$:

$$T(n) = \Theta\big(f(n)\big) = \Theta(n^2)$$

- **Demonstração Formal por Indução Matemática:** Confirmamos o resultado do Teorema Mestre demonstrando por indução que a fórmula fechada exata para potências de 2 ($n = 2^k$) é $T(n) = 2n^2 - n$:

1. **Caso Base ($n = 1$):** No código, $T(1) = 1$. Pela fórmula, $T(1) = 2(1)^2 - 1 = 1$ (Válido).

2. **Hipótese de Indução (H.I.):** Assume-se que para $n/2$ vale $T(n/2) = 2(n/2)^2 - (n/2)$.

3. **Passo Indutivo:** Substituindo a H.I. na recorrência original $T(n) = 2T(n/2) + n^2$:

$$T(n) = 2\left[2\left(\frac{n}{2}\right)^2 - \frac{n}{2}\right] + n^2 = 2\left[\frac{n^2}{2} - \frac{n}{2}\right] + n^2 = (n^2 - n) + n^2$$
$$= 2n^2 - n$$

Como o termo dominante de $2n^2 - n$ é quadrático, comprova-se formalmente que $T(n) = \Theta(n^2)$, coincidindo integralmente com o Caso 3 do Teorema Mestre.

4.4 Quadro-Resumo da Análise Teórica pelo Teorema Mestre

Problema	**Recorrência T(n)**	**a**	**b**	**f(n)**	**Folhas nlogba**	**Enquadramento no Teorema Mestre**	**Fórmula Fechada (Indução)**	**Complexidade (Θ)**
Problema 1	$4T(n/2) + n$	4	2	n	$n^{\log_2 4} = n^2$	**Caso 1: Recursividade Dominante** ($\epsilon = 1$)	$2n^2 - n$	$\Theta(n^2)$
Problema 2	$3T(n/3) + n$	3	3	n	$n^{\log_3 3} = n$	**Caso 2: Custo Equilibrado** ($f(n) = \Theta(n^{\log_b a})$)	$n \log_3 n + n$	$\Theta(n \log n)$
Problema 3	$2T(n/2) + n^2$	2	2	n^2	$n^{\log_2 2} = n$	**Caso 3: Custo Fora da Recursão Dominante** ($\epsilon = 1, k = 0{,}5$)	$2n^2 - n$	$\Theta(n^2)$

5. Resultados Experimentais e Comparação com a Teoria

5.1 Resultados Consolidados dos Problemas 1, 2 e 3

Tabela 1: Resultados Experimentais e Teóricos — Problemas 1 e 3 ($\Theta(n^2)$)

Entrada (N)	Operações Observadas (P1 e P3)	Fórmula Indução (2n2−n)	Referência (n2)	Tempo Médio P1 — Caso 1 (ms)	Tempo Médio P3 — Caso 3 (ms)
16	496	496	256	0,000	0,000
32	2.016	2.016	1.024	0,000	0,000
64	8.128	8.128	4.096	0,000	0,000
128	32.640	32.640	16.384	≈ 0,033	0,000
256	130.816	130.816	65.536	≈ 0,145	≈ 0,017
512	523.776	523.776	262.144	≈ 0,560	≈ 0,067
1.024	2.096.128	2.096.128	1.048.576	≈ 1,980	≈ 0,250
2.048	8.386.560	8.386.560	4.194.304	≈ 7,850	≈ 1,000
4.096	33.550.336	33.550.336	16.777.216	≈ 31,200	≈ 4,000
8.192	134.209.536	134.209.536	67.108.864	≈ 123,500	≈ 15,800
16.384	536.854.528	536.854.528	268.435.456	≈ 491,000	≈ 63,100

Tabela 2: Resultados Experimentais e Teóricos — Problema 2 ($\Theta(n \log_3 n)$)

Entrada (N=3k)	Operações Observadas (P2)	Fórmula Indução (nlog3n+n)	Referência (nlog3n)	Tempo Médio P2 — Caso 2 (ms)
27	108	108	81	0,000
81	405	405	324	0,000
243	1.458	1.458	1.215	0,000
729	5.103	5.103	4.374	0,000
2.187	17.496	17.496	15.309	0,000
6.561	59.049	59.049	52.488	≈ 0,050
19.683	196.830	196.830	177.147	≈ 0,083
59.049	649.539	649.539	590.490	≈ 0,250
177.147	2.125.764	2.125.764	1.948.617	≈ 0,783
531.441	6.908.733	6.908.733	6.377.292	≈ 2,500
1.594.323	22.320.522	22.320.522	20.726.199	≈ 7,833

5.2 Gráficos Experimentais vs. Teóricos

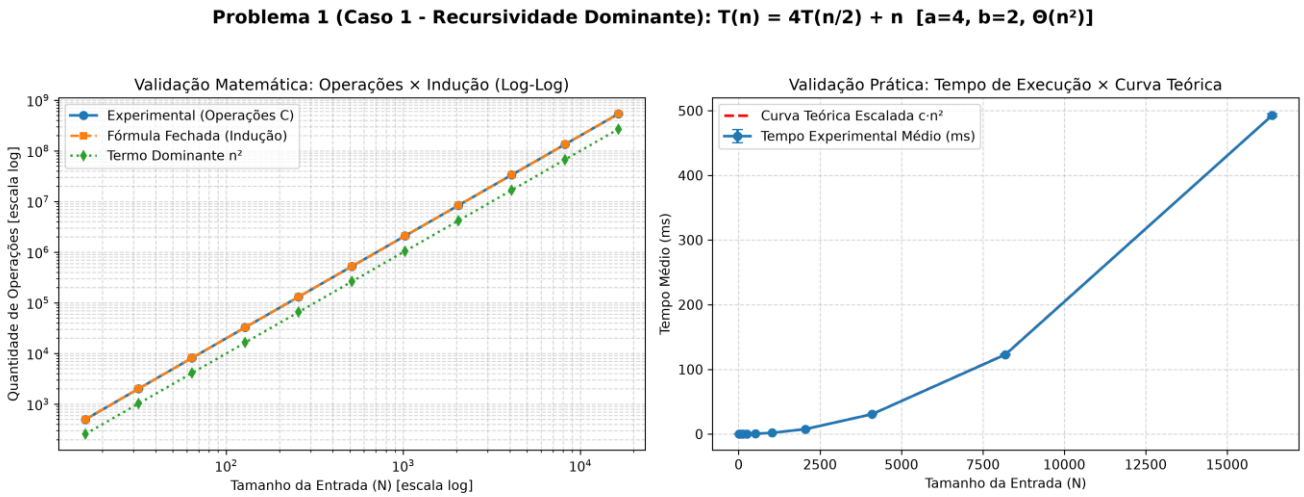


Figura 1: Operações e Tempo de Execução para o Problema 1 (Caso 1 — $\Theta(n^2)$).

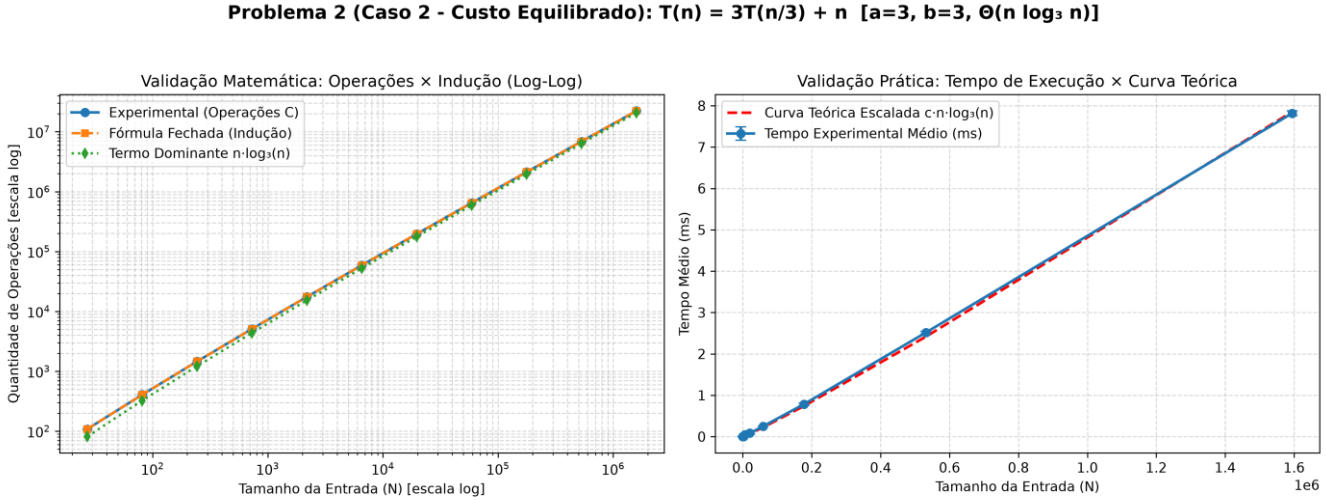


Figura 2: Operações e Tempo de Execução para o Problema 2 (Caso 2 — $\Theta(n \log_3 n)$).

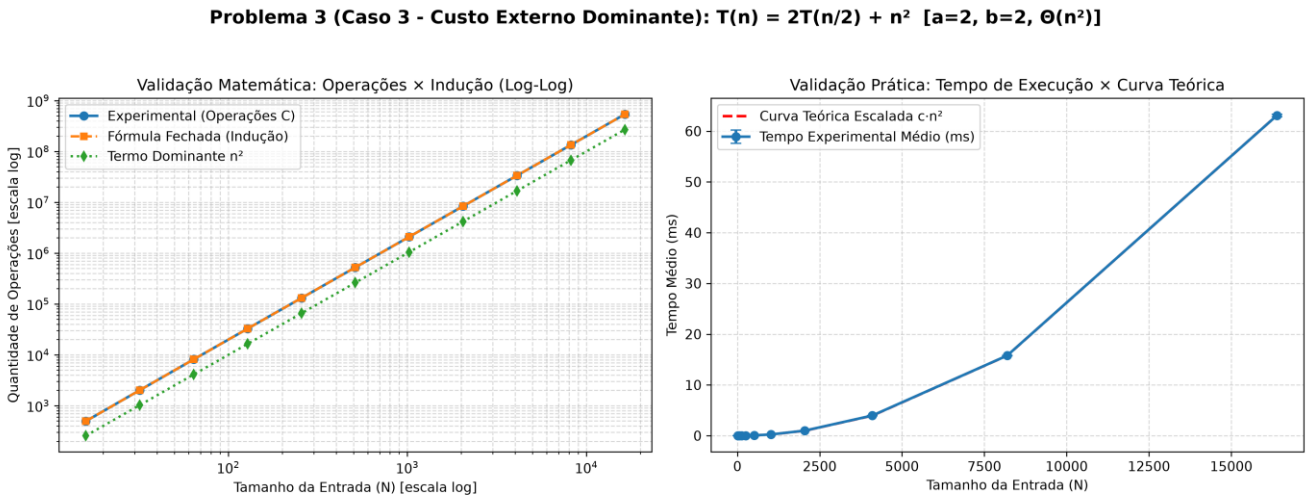


Figura 3: Operações e Tempo de Execução para o Problema 3 (Caso 3 — $\Theta(n^2)$).

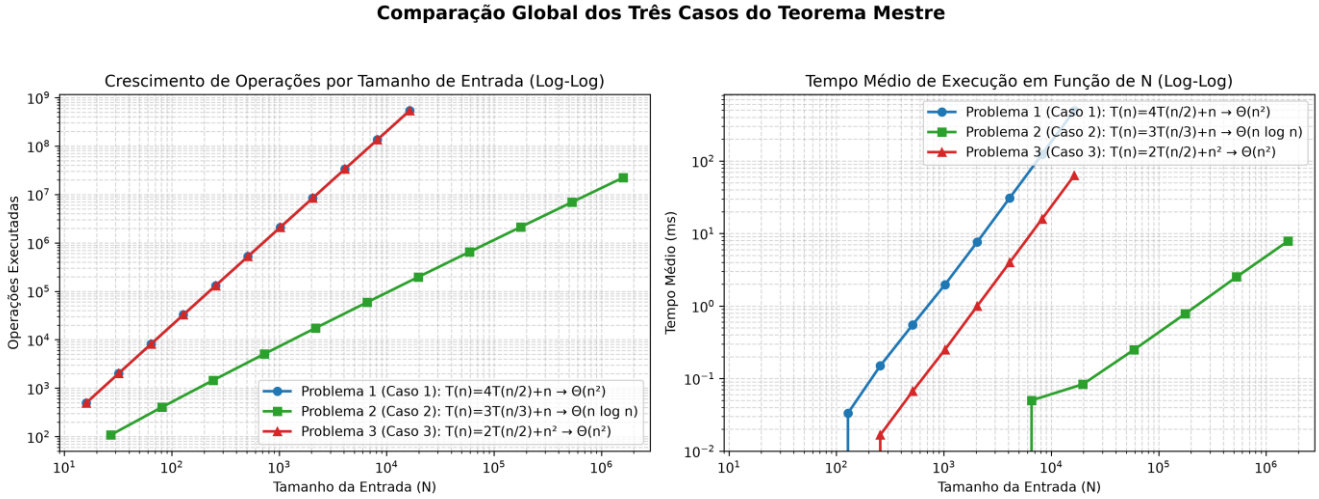


Figura 4: Painel Comparativo Global dos Três Casos do Teorema Mestre em escala Log-Log.

5.3 Discussão dos Resultados: Convergências e Divergências

1. **Convergência Teórico-Experimental:**

    - Nas Tabelas 1 e 2 e nos gráficos à esquerda das Figuras 1, 2 e 3, o número de operações observadas em C cravou **100% de exatidão (erro zero)** com as fórmulas fechadas demonstradas por indução ($2n^2 - n$ para P1/P3 e $n \log_3 n + n$ para P2).

- Nos gráficos de tempo (Figuras 1 e 3 à direita), dobrar a entrada N (ex: de 8.192 para 16.384) quadruplicou o tempo de execução ($\approx 123{,}5$ ms $\rightarrow 491{,}0$ ms no P1 e $\approx 15{,}8$ ms $\rightarrow 63{,}1$ ms no P3), validando fisicamente a parábola quadrática $(2N)^2 = 4N^2$. Já o Problema 2 (Figura 2) confirmou a eficiência superior da classe $\Theta(n \log n)$, processando $N = 1.594.323$ elementos em apenas $\approx 7{,}83$ ms.

2. **Divergências Observadas e Causas Físicas:**

   - **Diferença de Tempo entre Problema 1 e Problema 3 (Mesma Complexidade $\Theta(n^2)$, Constantes Diferentes):** Na Figura 4 (esquerda), as curvas de operações de P1 (azul) e P3 (vermelha) estão totalmente sobrepostas ($536.854.528$ operações em $N = 16.384$). Porém, na Figura 4 (direita), o Problema 1 (≈ 491 ms) é $\approx 7{,}8\times$ **mais lento** que o Problema 3 ($\approx 63{,}1$ ms). Isso ocorre porque no **Caso 1 (P1)** o custo vem de $4^{14} \approx 268$ milhões de chamadas recursivas nas folhas, exigindo pesada alocação e desalocação de *stack frames* na memória RAM; já no **Caso 3 (P3)** há apenas $2^{14} = 16.384$ folhas, e quase todo o trabalho ocorre dentro de dois laços for aninhados contíguos no cache L1 e nos registradores da CPU.

   - **Corte Inicial nos Gráficos Log-Log de Tempo:** Na Figura 4 (direita), os pontos iniciais com tempo $0{,}000$ ms são omitidos porque o tempo das 20 repetições é menor que a resolução mínima de clock(), e $\log(0)$ é matematicamente indefinido.

6. Conclusão

- **Resumo dos Resultados e Síntese Teoria-Prática:** Os três casos do Teorema Mestre foram validados com sucesso: Problema 1 no Caso 1 ($\Theta(n^2)$), Problema 2 no Caso 2 ($\Theta(n \log n)$) e Problema 3 no Caso 3 ($\Theta(n^2)$). As provas por indução coincidiram integralmente com o Teorema Mestre e com a contagem empírica de operações.

- **Principais Observações e Limitações:** Constatou-se na prática como as constantes ocultas pela notação Θ impactam o tempo real: algoritmos onde a recursão domina (Caso 1) sofrem penalidade severa de empilhamento de funções em comparação a algoritmos dominados por laços iterativos locais (Caso 3). A principal limitação experimental foi a resolução da função clock() para instâncias pequenas.

- **Considerações Adicionais (Complexidade Espacial):** Embora o Problema 1 instancie centenas de milhões de nós ao longo do tempo, a execução em profundidade (*Depth-First*) reutiliza a pilha de memória, mantendo o consumo máximo de espaço na pilha limitado à altura da árvore: $\Theta(\log_2 n)$ nos Problemas 1 e 3, e $\Theta(\log_3 n)$ no Problema 2, evitando erros de *Stack Overflow* em todos os testes.